

SUEÑATOR S.L.

Proyecto Intermodular 1º DAW

Análisis del proyecto y Guía de conexión JDBC

Abril 2026

1. Análisis del estado actual del proyecto

Se ha revisado el repositorio suenhatorSL y el código fuente del proyecto. A continuación se detalla el estado de cada entregable requerido por módulo.

1.1 Módulo: Lenguajes de Marcas (0373) — HTML + CSS

Elemento	Estado	Qué falta / Qué hacer
Página de inicio (index.html)	✔Hecho	—
Páginas adicionales (contacto, packs, precios, sobreNosotros)	✔Hecho	—
Estructura semántica HTML	✔Hecho	—
Navegación funcional	✔Hecho	—
Diseño visual CSS (suenhator.css)	✔Hecho	—
Responsive básico	❑ Parcial	Revisar que haya media queries en el CSS
README explicando la web	✔Hecho	—

1.2 Módulo: Bases de Datos (0484)

Elemento	Estado	Qué falta / Qué hacer
Análisis de datos / empresa ficticia	✔Hecho	—
Diagrama E/R (ER_suenhator.drawio)	✔Hecho	Exportar a PDF o imagen para el entregable
Modelo relacional (modelo_relacional.drawio)	✔Hecho	Exportar a PDF o imagen para el entregable

Script creación de tablas (creacion_suenhatorDB.sql)	✓ Hecho	—
Script inserción de datos (insercion_suenhator.sql)	✓ Hecho	—
Script de consultas (queries.sql)	✓ Hecho	—
README de BBDD explicando el diseño	✗ Falta	Crear README en DOCUMENTACION_BD/

1.3 Módulo: Programación (0485)

Elemento	Estado	Qué falta / Qué hacer
Clases modelo (Cliente, Reserva, Pack, etc.)	✓ Hecho	—
Controllers (ClienteController, CompraController, ReservaController)	✓ Hecho	—
Interfaz JavaFX (vistas FXML + ViewController)	✓ Hecho	—
Dataset en memoria (Dataset.java)	✓ Hecho	Funciona en memoria, falta JDBC
Conexión JDBC a la base de datos	✗ FALTA	Es obligatorio — ver Sección 2 de este doc
Operaciones CRUD reales con BBDD	✗ FALTA	Depende de JDBC — ver Sección 2
Dependencia MySQL en pom.xml	✗ FALTA	Hay que añadirla — ver Sección 2, Paso 1
module-info.java actualizado para JDBC	✗ FALTA	Hay que añadir requires java.sql — ver Sección 2
Gestión de errores (try/catch SQLException)	□ Parcial	Hay try/catch en vistas, falta en BBDD
README de programación	□ Parcial	Ampliar con instrucciones de ejecución y JDBC

1.4 Módulo: Entornos de Desarrollo (0487)

Elemento	Estado	Qué falta / Qué hacer
Repositorio GitHub estructurado	✓ Hecho	—
Carpetas organizadas (/docs, /DOCUMENTACION_BD, etc.)	✓ Hecho	—
README principal del proyecto	✓ Hecho	Ampliar con información de todos los módulos
Historial de commits coherente y progresivo	□ Parcial	Revisar que haya commits frecuentes y descriptivos
Archivos .gitignore correctos	✓ Hecho	—

Sin archivos basura en el repo	❏ Parcial	Hay archivos .bkp de drawio, valorar limpiarlos
--------------------------------	------------------	---

1.5 Módulo: Sistemas Informáticos (0483)

Elemento	Estado	Qué falta / Qué hacer
Informe técnico de entorno de ejecución	XFALTA	Crear /docs/sistemas/ con el informe
Requisitos de hardware y SO	XFALTA	Incluir en el informe técnico
Guía de instalación paso a paso	XFALTA	Incluir en el informe técnico
Usuarios, permisos y estructura de carpetas	XFALTA	Incluir en el informe técnico
Capturas de funcionamiento	XFALTA	Añadir screenshots de la app ejecutándose
Plan de mantenimiento	XFALTA	Incluir en el informe técnico

1.6 Módulo: Itinerario Empleabilidad (1709)

Elemento	Estado	Qué falta / Qué hacer
Perfil profesional personal	XFALTA	Crear en /docs/empleabilidad/
Presentación del proyecto para entrevista	XFALTA	Crear en /docs/empleabilidad/
Portfolio básico	XFALTA	Puede estar en README o PDF
Reflexión personal	XFALTA	Crear en /docs/empleabilidad/

2. Guía completa: Cómo añadir la conexión JDBC

Tu aplicación JavaFX usa actualmente el Dataset.java para almacenar todo en memoria. Los datos desaparecen cada vez que cierras la app. La conexión JDBC es OBLIGATORIA según la rúbrica (vale 2 puntos directos). Aquí tienes cómo hacerlo paso a paso.

¿Qué es JDBC?

JDBC (Java Database Connectivity) es la API estándar de Java que permite conectar tu programa con una base de datos. Con JDBC puedes ejecutar consultas SQL desde tu código Java y obtener los resultados.

Paso 1 — Añadir la dependencia MySQL en pom.xml

Tu pom.xml actual tiene JavaFX y Lombok, pero NO tiene el conector de MySQL. Sin él, Java no puede hablar con tu base de datos. Abre tu pom.xml y añade esto dentro de <dependencies>:

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.3.0</version>
</dependency>
```

Después, en IntelliJ: haz clic derecho sobre el pom.xml → Maven → Reload Project. Espera a que descargue el conector.

Paso 2 — Actualizar module-info.java

Tu proyecto usa módulos Java (tiene un module-info.java). Tienes que declarar que necesitas el módulo java.sql, que es el que contiene JDBC. Abre el archivo y añade la línea resaltada:

```
module org.example.suenhator {
    requires javafx.controls;
    requires javafx.fxml;
    requires static lombok;
    requires java.sql;           // <-- AÑADE ESTA LÍNEA
    requires model;             // si tu módulo model está separado

    opens org.example.suenhator to javafx.fxml;
    exports org.example.suenhator;
    exports org.example.suenhator.controller;
    opens org.example.suenhator.controller to javafx.fxml;
}
```

Paso 3 — Crear la clase ConexionDB

Crea una nueva clase en el paquete org.example.suenhator.data (donde ya está Dataset.java). Llámala ConexionDB.java. Esta clase será la responsable de abrir y cerrar la conexión con MySQL:

```

package org.example.suenhator.data;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConexionDB {

    // Cambia estos valores por los de tu MySQL
    private static final String URL = "jdbc:mysql://localhost:3306/suenhator";
    private static final String USUARIO = "root";
    private static final String PASSWORD = "tu_contraseña";

    // Guarda la conexión abierta para reutilizarla
    private static Connection conexion = null;

    // Método para obtener la conexión (la abre si no está abierta)
    public static Connection getConexion() {
        try {
            if (conexion == null || conexion.isClosed()) {
                conexion = DriverManager.getConnection(URL, USUARIO, PASSWORD);
                System.out.println("Conexion con la BBDD establecida correctamente");
            }
        } catch (SQLException e) {
            System.err.println("Error al conectar con la BBDD: " + e.getMessage());
        }
        return conexion;
    }

    // Método para cerrar la conexión al salir de la app
    public static void cerrarConexion() {
        try {
            if (conexion != null && !conexion.isClosed()) {
                conexion.close();
                System.out.println("Conexion cerrada correctamente");
            }
        } catch (SQLException e) {
            System.err.println("Error al cerrar la conexion: " + e.getMessage());
        }
    }
}

```

Importante: nombre de la base de datos

La URL 'jdbc:mysql://localhost:3306/suenhator' asume que tu base de datos se llama 'suenhator'. Si la creaste con otro nombre en tu script SQL, cámbialo aquí. El puerto 3306 es el que usa MySQL por defecto.

Paso 4 — Crear un DAO para Cliente

Un DAO (Data Access Object) es una clase que concentra todas las operaciones de base de datos para una entidad. En lugar de meter SQL en tu ClienteController, lo pones en un DAO limpio. Crea ClienteDAO.java en el paquete org.example.suenhator.data:

```
package org.example.suenhator.data;

import model.Cliente;
import java.sql.*;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

public class ClienteDAO {

    // LEER: obtener todos los clientes de la BBDD
    public List<Cliente> obtenerTodos() {
        List<Cliente> lista = new ArrayList<>();
        String sql = "SELECT * FROM cliente";

        try (Connection con = ConexionDB.getConnection();
            Statement stmt = con.createStatement();
            ResultSet rs = stmt.executeQuery(sql)) {

            while (rs.next()) {
                Cliente c = new Cliente();
                c.setIdCliente(rs.getInt("id_cliente"));
                c.setNombre(rs.getString("nombre"));
                c.setApellidos(rs.getString("apellidos"));
                c.setDni(rs.getString("dni"));
                c.setTelefono(rs.getString("telefono"));
                c.setEmail(rs.getString("email"));
                // Convertir Date de SQL a LocalDate de Java
                if (rs.getDate("fecha_alta") != null)
                    c.setFechaAlta(rs.getDate("fecha_alta").toLocalDate());
                if (rs.getDate("fecha_nac") != null)
                    c.setFechaNac(rs.getDate("fecha_nac").toLocalDate());
                lista.add(c);
            }
        } catch (SQLException e) {
            System.err.println("Error al obtener clientes: " + e.getMessage());
        }
        return lista;
    }

    // CREAR: insertar un nuevo cliente en la BBDD
    public boolean insertar(Cliente c) {
        String sql = "INSERT INTO cliente (nombre, apellidos, dni, telefono, " +
            "email, password, fecha_alta, fecha_nac) " +
            "VALUES (?, ?, ?, ?, ?, ?, ?, ?)";

        try (Connection con = ConexionDB.getConnection();
            PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setString(1, c.getNombre());
            ps.setString(2, c.getApellidos());
```

```

        ps.setString(3, c.getDni());
        ps.setString(4, c.getTelefono());
        ps.setString(5, c.getEmail());
        ps.setString(6, "password123"); // en un proyecto real: hashear
        ps.setDate(7, Date.valueOf(c.getFechaAlta() != null ?
            c.getFechaAlta() : LocalDate.now()));
        ps.setDate(8, c.getFechaNac() != null ?
            Date.valueOf(c.getFechaNac()) : null);

        int filas = ps.executeUpdate();
        return filas > 0;

    } catch (SQLException e) {
        System.err.println("Error al insertar cliente: " + e.getMessage());
        return false;
    }
}

// ELIMINAR: borrar un cliente por su DNI
public boolean eliminar(String dni) {
    String sql = "DELETE FROM cliente WHERE dni = ?";

    try (Connection con = ConexionDB.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, dni);
        int filas = ps.executeUpdate();
        return filas > 0;

    } catch (SQLException e) {
        System.err.println("Error al eliminar cliente: " + e.getMessage());
        return false;
    }
}

// MODIFICAR: actualizar datos de un cliente
public boolean modificar(Cliente c) {
    String sql = "UPDATE cliente SET nombre=?, apellidos=?, telefono=?,
email=? " +
        "WHERE dni=?";

    try (Connection con = ConexionDB.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, c.getNombre());
        ps.setString(2, c.getApellidos());
        ps.setString(3, c.getTelefono());
        ps.setString(4, c.getEmail());
        ps.setString(5, c.getDni());

        int filas = ps.executeUpdate();
        return filas > 0;

    } catch (SQLException e) {
        System.err.println("Error al modificar cliente: " + e.getMessage());
    }
}

```

```

        return false;
    }
}

// BUSCAR por DNI
public Cliente buscarPorDni(String dni) {
    String sql = "SELECT * FROM cliente WHERE dni = ?";

    try (Connection con = ConexionDB.getConexion();
        PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, dni);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                Cliente c = new Cliente();
                c.setIdCliente(rs.getInt("id_cliente"));
                c.setNombre(rs.getString("nombre"));
                c.setApellidos(rs.getString("apellidos"));
                c.setDni(rs.getString("dni"));
                c.setTelefono(rs.getString("telefono"));
                c.setEmail(rs.getString("email"));
                return c;
            }
        }
    } catch (SQLException e) {
        System.err.println("Error al buscar cliente: " + e.getMessage());
    }
    return null;
}
}

```

Paso 5 — Conectar el DAO con el Dataset y la interfaz

Ahora tienes que hacer que la app cargue los datos de la BBDD al arrancar, en lugar de usar listas vacías. Modifica la clase `HelloApplication.java` (el punto de entrada de la app) para que cargue los datos al inicio:

```

// En HelloApplication.java, en el método start() o init():

// 1. Crea el DAO
ClienteDAO clienteDAO = new ClienteDAO();

// 2. Carga todos los clientes de la BBDD en el Dataset
Dataset.listaClientes.setAll(clienteDAO.obtenerTodos());

// Repite lo mismo para Pack, Reserva, etc. con sus DAOs

```

También tienes que actualizar `ClienteController.java` para que use el DAO en lugar de solo modificar la lista en memoria:

```

// En ClienteController.java, en el método darDeAlta():

ClienteDAO dao = new ClienteDAO();

```

```
// Guarda en BBDD
if (dao.insertar(cliente)) {
    // Si se guardó correctamente, también actualiza la lista en memoria
    Dataset.listaClientes.add(cliente);
    return cliente;
}

// En el método darDeBaja():
if (dao.eliminar(cliente.getDni())) {
    Dataset.listaClientes.remove(cliente);
    return true;
}
```

Paso 6 — Repetir el patrón DAO para el resto de entidades

Una vez que ClienteDAO funciona, repite exactamente el mismo patrón para las demás entidades que ya tienes en tu modelo. El proceso es siempre igual:

1. Crea la clase XxxDAO.java en el paquete data
2. Implementa los métodos obtenerTodos(), insertar(), modificar() y eliminar()
3. Adapta las columnas SQL a los atributos de tu clase modelo con los set correspondientes
4. Actualiza el Controller correspondiente para que llame al DAO

Entidad	DAO a crear	Tabla SQL
Cliente	ClienteDAO	cliente
Reserva	ReservaDAO	reserva
Pack	PackDAO	pack
Sala	SalaDAO	sala
Supervisor	SupervisorDAO	supervisor
Compra	CompraDAO	compra
Pago	PagoDAO	pago
Invitado	InvitadoDAO	invitado

Paso 7 — Probar la conexión

Antes de modificar toda la app, prueba que la conexión funciona con un test rápido. Puedes añadir este bloque temporalmente en tu HelloApplication.java o en cualquier sitio al arrancar:

```
// Test rápido de conexión — BORRA esto después de probarlo
import java.sql.Connection;
import org.example.suenhator.data.ConexionDB;

Connection con = ConexionDB.getConexion();
if (con != null) {
```

```
        System.out.println("CONEXION OK - JDBC funciona correctamente");  
    } else {  
        System.out.println("FALLO DE CONEXION - revisa URL, usuario y contraseña");  
    }  
}
```

Errores frecuentes al conectar

- 'Unknown database suenhator' → la BBDD no existe, ejecuta tu script creacion_suenhatorDB.sql primero en MySQL Workbench.
- 'Access denied for user root' → el password en ConexionDB.java no coincide con el de tu MySQL.
- 'Communications link failure' → MySQL no está arrancado. Arranca el servicio MySQL.
- SQLException: No suitable driver → falta la dependencia MySQL en pom.xml (vuelve al Paso 1).

3. Lista de pendientes por módulo (checklist)

Aquí tienes un resumen rápido de todo lo que queda por hacer, ordenado por prioridad y módulo.

☐ Urgente — JDBC (Programación 0485)

- Añadir dependencia mysql-connector-j en pom.xml
- Añadir requires java.sql en module-info.java
- Crear la clase ConexionDB.java
- Crear ClienteDAO.java con CRUD completo
- Conectar el DAO al Dataset y al ClienteController
- Repetir el patrón DAO para Reserva, Pack, Compra, Pago, Invitado
- Actualizar todos los controllers para usar los DAOs
- Probar que la app carga datos reales de MySQL

☐ Importante — Documentación pendiente

- **Exportar ER y modelo relacional a PDF o imagen para el entregable BBDD (0484):**
- **Crear README en /DOCUMENTACION_BD/ explicando el diseño de la BBDD BBDD (0484):**
- **Ampliar README con instrucciones de ejecución, dependencias y qué parte usa JDBC Programación (0485):**
- **Revisar historial de commits: que sean frecuentes, descriptivos y progresivos Entornos (0487):**

☐ Pendiente — Sistemas Informáticos (0483)

- Crear la carpeta /docs/sistemas/ en el repositorio
- Redactar el Informe Técnico de Entorno de Ejecución con: tipo de equipo, SO recomendado (Windows/Linux), requisitos mínimos de hardware, guía de instalación paso a paso (JDK 21+, Maven, MySQL, IntelliJ), usuarios y permisos de MySQL, plan de mantenimiento básico
- Añadir capturas de pantalla de la app funcionando

☐ Pendiente — Itinerario Empleabilidad (1709)

- Crear la carpeta /docs/empleabilidad/ en el repositorio
- Redactar el documento de perfil profesional personal
- Redactar la presentación del proyecto (qué es, qué problema resuelve, tecnologías)
- Crear un portfolio básico con capturas y enlace al repo
- Redactar la reflexión personal (qué has aprendido, qué mejorarías)

☐ Ya hecho — Solo revisión

- Web HTML/CSS (docs/) — revisar responsive con media queries
- README principal — ampliar mencionando todos los módulos

- Limpiar archivos .bkp de drawio del repositorio